

SQL Notes

Table Used: employees

empid	name	department	salary	age	city
1	Keerti	HR	30000	21	Bangalore
2	Rahul	IT	50000	22	Mysore
3	Anjel	Finance	45000	20	Hubli
4	Amit	Marketing	60000	23	Haveri
5	Riya	HR	35000	22	Bangalore
6	Karan	IT	55000	24	Mysore
7	Sneha	Finance	40000	21	Hubli
8	Arjun	Marketing	65000	25	Belgaum
9	Priya	HR	45000	23	Dharwad
10	Vikram	IT	70000	26	Haveri

1. SELECT : The SELECT statement is used to retrieve data from a table.

Top 5 Most Asked Interview Questions

Question 1: Show all employee details.

```
SELECT *  
FROM employees;
```

Question 2: Display employee names and salaries.

```
SELECT name,salary  
FROM employees;
```

Question 3 : Display employee ID, name, and department.

```
SELECT empid,name,department  
FROM employees;
```

Question 5 : Display all employee information required for HR analysis.

```
SELECT empid,name,department,salary,age,city  
FROM employees;
```

2. WHERE : The WHERE clause is used to filter records based on a specified condition.

Top 5 Most Asked Interview Questions

Question 1 : Find all employees who work in Bangalore.

```
SELECT * FROM employees WHERE city = 'Bangalore';
```

Question 2 : Find employees whose salary is greater than 50000.

```
SELECT name, salary FROM employees WHERE salary > 50000;
```

Question 3 : Find employees whose age is less than 23.

```
SELECT name, age FROM employees WHERE age < 23;
```

3.AND,OR,NOT : AND, OR, and NOT are used to apply multiple conditions while filtering data. **Important Notes**

AND : Returns records only when all conditions are true.

OR : Returns records when at least one condition is true.

NOT : Returns records that do not satisfy the condition

Top 5 Most Asked Interview Questions

Question 1 : Find employees who work in the IT department and live in Mysore.

```
SELECT * FROM employees WHERE department = 'IT' AND city = 'Mysore';
```

Question 2 : Find employees whose salary is greater than 50000 and age is less than 25.

```
SELECT name, salary, age FROM employees WHERE salary > 50000 AND age < 25;
```

Question 3 : Find employees who live in Bangalore or Mysore.

```
SELECT * FROM employees WHERE city = 'Bangalore' OR city = 'Mysore'
```

Question 4 : Find employees who belong to the HR department or Finance department.

```
SELECT name, department FROM employees WHERE department = 'HR' OR department = 'Finance';
```

Question 5 : Find employees who do not belong to the HR department.

```
SELECT * FROM employees WHERE NOT department = 'HR';
```

3. ORDER BY : ORDER BY is used to sort the result set in ascending or descending order.

Top 5 Most Asked Interview Questions

Question 1 : Display all employees sorted by salary in ascending order.

SELECT * FROM employees ORDER BY salary;

Question 2 : Display all employees sorted by salary in descending order.

SELECT * FROM employees ORDER BY salary DESC;

Question 3 : Display employee names and ages sorted by age in descending order.

SELECT name, age FROM employees ORDER BY age DESC;

Question 4 : Display all employees sorted by department in ascending order and salary in descending order.

SELECT * FROM employees ORDER BY department ASC, salary DESC;

Question 5 : Display employee names, cities, and salaries sorted by city and then by salary.

SELECT name, city, salary FROM employees ORDER BY city ASC, salary DESC;

4. LIMIT : LIMIT is used to restrict the number of rows returned by a query.

Top 5 Most Asked Interview Questions

Question 1 : Display the first 3 employees from the employees table.

SELECT * FROM employees LIMIT 3;

Question 2 : Display the top 5 employees from the employees table.

SELECT * FROM employees LIMIT 5;

Question 3 : Display the 3 highest-paid employees.

SELECT * FROM employees ORDER BY salary DESC LIMIT 3;

Question 4 : Display the 2 youngest employees.

SELECT * FROM employees ORDER BY age LIMIT 2;

Question 5 : Display employee names and salaries of the top 4 highest-paid employees.

SELECT name, salary FROM employees ORDER BY salary DESC LIMIT 4;

5. DISTINCT : DISTINCT is used to return only unique values and remove duplicate values from the result.

Top 5 Most Asked Interview Questions

Question 1 : Display all unique cities where employees are located.

```
SELECT DISTINCT city FROM employees;
```

Question 2 : Display all unique departments in the company.

```
SELECT DISTINCT department FROM employees;
```

Question 3 : Display all unique salary values.

```
SELECT DISTINCT salary FROM employees;
```

Question 4 : Display all unique combinations of city and department.

```
SELECT DISTINCT city, department FROM employees;
```

Question 5 : Display all unique cities in descending order.

```
SELECT DISTINCT city FROM employees ORDER BY city DESC;
```

6. ALIASES : Aliases are used to give temporary names to columns or tables to make the output more readable.

Top 5 Most Asked Interview Questions

Question 1 : Display employee names with the column name Employee_Name.

```
SELECT name AS Employee_Name FROM employees;
```

Question 2 : Display salaries with the column name Monthly_Salary.

```
SELECT salary AS Monthly_Salary FROM employees;
```

Question 3 : Display department as Dept and city as Location.

```
SELECT department AS Dept, city AS Location FROM employees;
```

Question 4 : Display employee name, salary, and age with meaningful column names.

```
SELECT name AS Employee_Name, salary AS Salary_Amount, age AS Employee_Age  
FROM employees;
```

Question 5 : Display employee ID, name, and department using aliases.

```
SELECT empid AS Employee_ID, name AS Employee_Name, department AS Dept FROM  
employees;
```

7. COUNT() : COUNT() is used to count the number of rows or non-NULL values in a column.

Top 5 Most Asked Interview Questions

Question 1 : Find the total number of employees in the company.

```
SELECT COUNT(empid) FROM employees;
```

Question 2 : Find the number of employees working in the IT department.

```
SELECT COUNT(empid) FROM employees WHERE department = 'IT';
```

Question 3 : Find the number of employees living in Bangalore.

```
SELECT COUNT(empid) FROM employees WHERE city = 'Bangalore';
```

Question 4 : Find the number of employees whose salary is greater than 50000.

```
SELECT COUNT(empid) FROM employees WHERE salary > 50000;
```

Question 5 : Find the number of unique departments in the company.

```
SELECT COUNT(DISTINCT department) FROM employees;
```

8. SUM() and AVG() : SUM() is used to calculate the total of a numeric column, and AVG() is used to calculate the average value of a numeric column.

Top 5 Most Asked Interview Questions

Question 1 : Find the total salary paid to all employees.

```
SELECT SUM(salary) FROM employees;
```

Question 2 : Find the total salary paid to employees in the IT department.

```
SELECT SUM(salary) FROM employees WHERE department = 'IT';
```

Question 3 : Find the average salary of all employees.

```
SELECT AVG(salary) FROM employees;
```

Question 4 : Find the average salary of employees working in the Finance department.

```
SELECT AVG(salary) FROM employees WHERE department = 'Finance';
```

Question 5 : Find the average age of all employees.

```
SELECT AVG(age) FROM employees;
```

9. MIN() and MAX() : MIN() is used to find the smallest value, and MAX() is used to find the largest value in a column.

Top 5 Most Asked Interview Questions

Question 1 : Find the minimum salary in the company.

```
SELECT MIN(salary) FROM employees;
```

Question 2 : Find the maximum salary in the company.

```
SELECT MAX(salary) FROM employees;
```

Question 3 : Find the minimum age among all employees.

```
SELECT MIN(age) FROM employees;
```

Question 4 : Find the maximum salary in the IT department.

```
SELECT MAX(salary) FROM employees WHERE department = 'IT';
```

Question 5 : Find the minimum salary in the HR department.

```
SELECT MIN(salary) FROM employees WHERE department = 'HR';
```

10. GROUP BY : GROUP BY is used to group rows that have the same values into summary rows and is commonly used with aggregate functions.

Top 5 Most Asked Interview Questions

Question 1 : Find the total salary paid in each department.

```
SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department;
```

Question 2 : Find the average salary in each department.

```
SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department;
```

Question 3 : Find the number of employees in each department.

```
SELECT department, COUNT(empid) AS employee_count FROM employees GROUP BY department;
```

Question 4 : Find the number of employees in each city.

```
SELECT city, COUNT(empid) AS employee_count FROM employees GROUP BY city;
```

Question 5 : Find the average salary for each department in each city.

```
SELECT department, city, AVG(salary) AS avg_salary FROM employees GROUP BY department, city;
```

11. HAVING : HAVING is used to filter grouped data after the GROUP BY clause is applied.

Top 5 Most Asked Interview Questions

Question 1 : Find departments whose total salary is greater than 100000.

```
SELECT department, SUM(salary) AS total_salary FROM employees GROUP BY department HAVING SUM(salary) > 100000;
```

Question 2 : Find cities whose average salary is greater than 45000.

```
SELECT city, AVG(salary) AS avg_salary FROM employees GROUP BY city HAVING AVG(salary) > 45000;
```

Question 3 : Find departments having more than 2 employees.

```
SELECT department, COUNT(empid) AS employee_count FROM employees GROUP BY department HAVING COUNT(empid) > 2;
```

Question 4 : Find cities where the minimum salary is greater than 35000.

```
SELECT city, MIN(salary) AS min_salary FROM employees GROUP BY city HAVING MIN(salary) > 35000;
```

Question 5 : Find department-city combinations whose average salary is greater than 45000.

```
SELECT department, city, AVG(salary) AS avg_salary FROM employees GROUP BY department, city HAVING AVG(salary) > 45000;
```

12. JOINS : JOIN is used to combine data from two or more tables based on a related column between them.

One-Line Interview Answer : JOIN is used to retrieve related data from multiple tables using a common column.

Table 1 : employees

empid	name	department	salary
1	Keerti	HR	30000
2	Rahul	IT	50000
3	Anjel	Finance	45000
4	Amit	Marketing	60000
5	Vikram	IT	70000

Table 2 : departments

department	manager		
HR	Anita		
IT	Ramesh		
Finance	Suresh		
Marketing	Priya		
Admin	Mahesh		

Common Column : department

employees.department = departments.department

Types of Joins

1. INNER JOIN
2. LEFT JOIN
3. RIGHT JOIN
4. FULL JOIN

13. INNER JOIN : INNER JOIN returns only the matching records from both tables based on a common column.

Top 5 Most Asked Interview Questions

Question 1 : Display employee name, department, and manager name.

```
SELECT e.name, d.department, d.manager  
FROM employees e  
INNER JOIN departments d  
ON e.department = d.department;
```

Question 2 : Display employee name, salary, and manager name.

```
SELECT e.name, e.salary, d.manager FROM employees e INNER JOIN departments d ON  
e.department = d.department;
```

Question 3 : Display employee name, city, department, and manager name.

```
SELECT e.name, e.city, d.department, d.manager FROM employees e INNER JOIN  
departments d ON e.department = d.department;
```

Question 4 : Find the number of employees working under each manager.

```
SELECT d.manager, COUNT(e.empid) AS employee_count FROM employees e INNER  
JOIN departments d ON e.department = d.department GROUP BY d.manager;
```

Question 5 : Find the average salary of employees in each department along with the manager name.

```
SELECT d.department, d.manager, AVG(e.salary) AS avg_salary FROM employees e  
INNER JOIN departments d ON e.department = d.department  
GROUP BY d.department, d.manager;
```


14. LEFT JOIN : LEFT JOIN returns all records from the left table and the matching records from the right table. If there is no match, NULL values are returned for the right table columns.

Top 5 Most Asked Interview Questions

Question 1 : Display all departments along with employee names.

```
SELECT d.department, d.manager, e.name  
FROM departments d  
LEFT JOIN employees e  
ON d.department = e.department;
```

Question 2 : Display all departments, managers, and employee salaries.

```
SELECT d.department, d.manager, e.salary FROM departments d LEFT JOIN employees e  
ON d.department = e.department;
```

Question 3 : Find departments that do not have any employees.

```
SELECT d.department, d.manager FROM departments d LEFT JOIN employees e ON  
d.department = e.department WHERE e.empid IS NULL;
```

Question 4 : Find the number of employees in each department, including departments with no employees.

```
SELECT d.department, d.manager, COUNT(e.empid) AS employee_count FROM  
departments d LEFT JOIN employees e ON d.department = e.department GROUP BY  
d.department, d.manager;
```

Question 5 : Find the average salary in each department, including departments with no employees.

```
SELECT d.department, d.manager, AVG(e.salary) AS avg_salary FROM departments d  
LEFT JOIN employees e ON d.department = e.department GROUP BY d.department,  
d.manager;
```

15. RIGHT JOIN : RIGHT JOIN returns all records from the right table and the matching records from the left table. If there is no match, NULL values are returned for the left table columns.

Top 5 Most Asked Interview Questions

Question 1 : Display all departments along with employee names.

```
SELECT e.name, d.department, d.manager  
FROM employees e
```

RIGHT JOIN departments d

ON e.department = d.department;

Question 2 : Display all departments and their managers, even if no employees exist in that department.

```
SELECT d.department, d.manager, e.name FROM employees e RIGHT JOIN departments d
ON e.department = d.department;
```

Question 3 : Find departments that do not have any employees.

```
SELECT d.department, d.manager FROM employees e RIGHT JOIN departments d ON
e.department = d.department WHERE e.empid IS NULL;
```

Question 4 : Find the number of employees under each manager, including managers with no employees.

```
SELECT d.manager, COUNT(e.empid) AS employee_count FROM employees e RIGHT
JOIN departments d ON e.department = d.department GROUP BY d.manager;
```

Question 5 : Find the average salary of employees under each manager, including managers with no employees.

```
SELECT d.manager, AVG(e.salary) AS avg_salary FROM employees e RIGHT JOIN
departments d ON e.department = d.department GROUP BY d.manager;
```

16. FULL JOIN : FULL JOIN returns all records from both tables. If there is no match in either table, NULL values are returned for the missing side.

Top 5 Most Asked Interview Questions

Question 1 : Display all employees, departments, and managers, including unmatched records from both tables.

```
SELECT e.name, d.department, d.manager
FROM employees e
LEFT JOIN departments d
ON e.department = d.department
UNION SELECT e.name, d.department, d.manager
FROM employees e
RIGHT JOIN departments d
ON e.department = d.department;
```

Question 2 : Display all departments and employee names, including departments without employees and employees without departments.

```
SELECT d.department, e.name FROM departments d LEFT JOIN employees e ON  
d.department = e.department UNION SELECT d.department, e.name FROM departments d  
RIGHT JOIN employees e ON d.department = e.department;
```

Question 3 : Find all unmatched records from both tables.

```
SELECT e.name, d.department, d.manager FROM employees e LEFT JOIN departments d  
ON e.department = d.department WHERE d.department IS NULL UNION SELECT e.name,  
d.department, d.manager FROM employees e RIGHT JOIN departments d ON e.department  
= d.department WHERE e.empid IS NULL;
```

Question 4 : Display all departments, managers, and employee salaries, including unmatched records.

```
SELECT d.department, d.manager, e.salary FROM departments d LEFT JOIN employees e  
ON d.department = e.department UNION SELECT d.department, d.manager, e.salary FROM  
departments d RIGHT JOIN employees e ON d.department = e.department;
```

Question 5 : Find records where either the employee or manager information is missing.

```
SELECT e.name, d.department, d.manager FROM employees e LEFT JOIN departments d  
ON e.department = d.department WHERE d.manager IS NULL UNION SELECT e.name,  
d.department, d.manager FROM employees e RIGHT JOIN departments d ON e.department  
= d.department WHERE e.name IS NULL;
```

17. SELF JOIN : A SELF JOIN is a join where a table is joined with itself to compare rows within the same table.

18. UNION & UNION ALL : UNION combines the results of two or more SELECT statements and removes duplicate rows, whereas UNION ALL combines the results and keeps duplicate rows.

Top 5 Most Asked Interview Questions

Question 1 : Display all unique customer names from online_customers and store_customers.

```
SELECT customer_name  
FROM online_customers  
UNION SELECT customer_name  
FROM store_customers;
```

Question 2 : Display all customer names from online_customers and store_customers, including duplicates.

```
SELECT customer_name FROM online_customers UNION ALL SELECT customer_name  
FROM store_customers;
```

Question 3 : Display all unique cities from employees_2024 and employees_2025.

```
SELECT city FROM employees_2024 UNION SELECT city FROM employees_2025;
```

Question 4 : Display all cities from employees_2024 and employees_2025, including duplicates.

```
SELECT city FROM employees_2024 UNION ALL SELECT city FROM employees_2025;
```

Question 5 : Display all employee names from hr_employees and it_employees without duplicates.

```
SELECT name FROM hr_employees UNION SELECT name FROM it_employees;
```

Interview Difference :

- **UNION** → Removes duplicates.
- **UNION ALL** → Keeps duplicates.
- **UNION ALL** is faster because it does not check for duplicates.

19. CASE WHEN : CASE WHEN is used to apply conditional logic in SQL and return different values based on specified conditions.

Top 5 Most Asked Interview Questions

Question 1 : Categorize employees as High Salary or Low Salary based on salary.

```
SELECT name, salary,  
  
CASE WHEN salary >= 50000 THEN 'High Salary'  
  
ELSE 'Low Salary' END AS salary_category  
  
FROM employees;
```

Question 2 : Categorize employees as Senior or Junior based on age.

```
SELECT name, age, CASE WHEN age >= 23 THEN 'Senior' ELSE 'Junior' END AS  
age_group FROM employees;
```

Question 5 : Assign priority levels to departments.

```
SELECT name, department, CASE WHEN department = 'IT' THEN 'High' WHEN  
department = 'Marketing' THEN 'Medium' ELSE 'Low' END AS priority FROM employees;
```

Question 4 : Categorize salaries as Excellent, Good, or Average.

```
SELECT name, salary,  
CASE WHEN salary >= 60000 THEN 'Excellent'  
WHEN salary >= 50000 THEN 'Good'  
ELSE 'Average' END AS salary_level  
FROM employees;
```

20. SUBQUERIES : A subquery is a query written inside another query and is used to retrieve data that will be used by the main query.

Top 5 Most Asked Interview Questions

Question 1 : Find employees whose salary is greater than the average salary.

```
SELECT name, salary  
FROM employees  
WHERE salary >  
(  
SELECT AVG(salary)  
FROM employees  
);
```

Question 2 : Find the employee(s) with the highest salary.

```
SELECT name, salary FROM employees WHERE salary = (SELECT MAX(salary) FROM  
employees);
```

Question 3 : Find employees who work in the same city as Keerti.

```
SELECT name, city FROM employees WHERE city = (SELECT city FROM employees  
WHERE name = 'Keerti');
```

Question 4 : Find employees whose salary is greater than Rahul's salary.

```
SELECT name, salary FROM employees WHERE salary > (SELECT salary FROM  
employees WHERE name = 'Rahul');
```

Question 5 : Find employees who belong to departments having more than 2 employees.

```
SELECT name, department FROM employees WHERE department IN  
(SELECT department FROM employees GROUP BY department HAVING COUNT(*)
```

21. CTE (Common Table Expression) : A CTE is a temporary result set created using the WITH clause that can be referenced within a query.

Top 5 Most Asked Interview Questions

Question 1 : Create a CTE to display employees whose salary is greater than 50000.

```
WITH high_salary AS
(SELECT * FROM employees
WHERE salary > 50000)
SELECT * FROM high_salary;
```

Question 2 : Create a CTE to display only IT employees.

```
WITH it_employees AS
(SELECT name, salary
FROM employees
WHERE department = 'IT')
SELECT * FROM it_employees;
```

Question 3 : Create a CTE to find the employee count in each city.

```
WITH city_count AS (SELECT city, COUNT(empid) AS employee_count FROM employees
GROUP BY city) SELECT * FROM city_count;
```

Question 4 : Create a CTE to find departments whose average salary is greater than 45000.

```
WITH dept_salary AS (SELECT department, AVG(salary) AS avg_salary FROM employees
GROUP BY department) SELECT * FROM dept_salary WHERE avg_salary > 45000;
```

Question 5 : Create a CTE to store the average salary and display employees earning more than the average salary.

```
WITH avg_salary_cte AS
(SELECT AVG(salary) AS avg_salary
FROM employees)
SELECT name, salary
FROM employees
WHERE salary >
(SELECT avg_salary
FROM avg_salary_cte);
```

22. ROW_NUMBER() : ROW_NUMBER() assigns a unique sequential number to each row based on the specified ordering.

Top 5 Most Asked Interview Questions

Question 1 : Assign row numbers to employees based on salary in descending order.

```
SELECT name, salary,  
ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM employees;
```

Question 2 : Find the employee with the highest salary using ROW_NUMBER().

```
WITH ranked AS (SELECT name, salary, ROW_NUMBER() OVER (ORDER BY salary  
DESC) AS row_num FROM employees) SELECT * FROM ranked WHERE row_num = 1;
```

Question 3 : Find the highest-paid employee in each department using ROW_NUMBER().

```
WITH dept_rank AS (SELECT name, department, salary, ROW_NUMBER() OVER  
(PARTITION BY department ORDER BY salary DESC) AS row_num FROM employees)  
SELECT * FROM dept_rank WHERE row_num = 1;
```

Question 4 : Display employees whose row number is between 3 and 5.

```
WITH ranked AS  
(SELECT name, salary, ROW_NUMBER() OVER (ORDER BY salary DESC) AS row_num  
FROM employees)  
SELECT * FROM ranked  
WHERE row_num BETWEEN 3 AND 5;
```

23. RANK() : RANK() assigns a rank to each row based on the specified ordering. If there is a tie, the same rank is assigned and the next rank is skipped.

Top 5 Most Asked Interview Questions

Question 1 : Assign ranks to employees based on salary in descending order.

```
SELECT name, salary,  
RANK() OVER (ORDER BY salary DESC) AS rank_num  
FROM employees;
```

Question 2 : Find the employee(s) with the highest salary using RANK().

```
WITH ranked AS (SELECT name, salary, RANK() OVER (ORDER BY salary DESC) AS  
rank_num FROM employees) SELECT * FROM ranked WHERE rank_num = 1;
```

Question .3 : Assign ranks within each department based on salary. SELECT name, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_num FROM employees;

Question 4 : Find the highest-paid employee in each department using RANK().

WITH dept_rank AS

(SELECT name, department, salary,

RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank_num
FROM employees)

SELECT * FROM dept_rank

WHERE rank_num = 1;

24. DENSE_RANK() : DENSE_RANK() assigns a rank to each row based on the specified ordering. If there is a tie, the same rank is assigned but the next rank is not skipped.

Top 5 Most Asked Interview Questions

Question 1 : Assign dense ranks to employees based on salary in descending order.

SELECT name, salary,

DENSE_RANK() OVER (ORDER BY salary DESC) AS dense_rank_num

FROM employees;

Question 2 : Find the employee(s) with the second highest salary using DENSE_RANK().

WITH ranked AS (SELECT name, salary, DENSE_RANK() OVER (ORDER BY salary
DESC) AS dense_rank_num FROM employees) SELECT * FROM ranked WHERE
dense_rank_num = 2;

Question 4 : Assign dense ranks within each department based on salary.

SELECT name, department, salary, DENSE_RANK() OVER (PARTITION BY department
ORDER BY salary DESC) AS dense_rank_num FROM employees;

Question 5 : Find the highest-paid employee in each department using DENSE_RANK().

WITH dept_rank AS

(SELECT name, department, salary,

DENSE_RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS
dense_rank_num

FROM employees)

SELECT * FROM dept_rank WHERE dense_rank_num = 1;

25. LAG() : LAG() returns the value from the previous row.

Top 2 Most Asked Interview Questions

Question 1 : Display the previous month's sales for each month.

```
SELECT month, sales,  
LAG(sales) OVER (ORDER BY month) AS previous_month_sales  
FROM monthly_sales;
```

Question 2 : Display the difference between current month sales and previous month sales.

```
SELECT month, sales,  
sales - LAG(sales) OVER (ORDER BY month) AS difference  
FROM monthly_sales;
```

26. LEAD() : LEAD() returns the value from the next row.

Top 2 Most Asked Interview Questions

Question 1 : Display the next month's sales for each month.

```
SELECT month, sales,  
LEAD(sales) OVER (ORDER BY month) AS next_month_sales  
FROM monthly_sales;
```

Question 2 : Display the difference between next month sales and current month sales.

```
SELECT month, sales,  
LEAD(sales) OVER (ORDER BY month) - sales AS difference  
FROM monthly_sales;
```

Interview Difference :

- **LAG()** → Returns data from the previous row.
- **LEAD()** → Returns data from the next row.

27. CREATE TABLE : CREATE TABLE is used to create a new table in a database.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table with employee ID, name, department, salary, age, and city.

```
CREATE TABLE employees (empid INT, name VARCHAR(50), department  
VARCHAR(50), salary INT, age INT, city VARCHAR(50));
```

Question 2 : Create an employees table where empid is the primary key.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), salary  
INT);
```

Question 3 : Create an employees table where name cannot be NULL.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50) NOT  
NULL, salary INT);
```

Question 4 : Create an employees table where email must be unique.

```
CREATE TABLE employees (empid INT PRIMARY KEY, email VARCHAR(100)  
UNIQUE);
```

Question 5 : Create an employees table where salary has a default value of 30000.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), salary  
INT DEFAULT 30000);
```

PRIMARY KEY : A PRIMARY KEY uniquely identifies each record in a table. It cannot contain duplicate or NULL values.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table with empid as the Primary Key.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), salary  
INT);
```

Question 2 : Create a students table with student_id as the Primary Key.

```
CREATE TABLE students (student_id INT PRIMARY KEY, name VARCHAR(50), marks  
INT);
```

DEFAULT : DEFAULT automatically assigns a value to a column when no value is provided.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table where salary defaults to 30000.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), salary INT DEFAULT 30000);
```

Question 2 : Create a students table where marks default to 0.

```
CREATE TABLE students (student_id INT PRIMARY KEY, name VARCHAR(50), marks INT DEFAULT 0);
```

NOT NULL : NOT NULL ensures that a column cannot contain NULL values.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table where name cannot be NULL.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50) NOT NULL);
```

Question 2 : Create an employees table where department cannot be NULL.

```
CREATE TABLE employees (empid INT PRIMARY KEY, department VARCHAR(50) NOT NULL);
```

UNIQUE : UNIQUE ensures that all values in a column are different.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table where email must be unique.

```
CREATE TABLE employees (empid INT PRIMARY KEY, email VARCHAR(100) UNIQUE);
```

Question 2 : Create a students table where roll number must be unique.

```
CREATE TABLE students (student_id INT PRIMARY KEY, roll_no INT UNIQUE);
```

AUTO_INCREMENT : AUTO_INCREMENT automatically generates sequential numbers for a column, usually the Primary Key.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table with AUTO_INCREMENT employee ID.

```
CREATE TABLE employees (empid INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50));
```

Question 2 : Create an employees table with AUTO_INCREMENT and NOT NULL name.

```
CREATE TABLE employees (empid INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50) NOT NULL);
```

Question 3 : Create an employees table with AUTO_INCREMENT, UNIQUE email, and DEFAULT salary.

```
CREATE TABLE employees (empid INT AUTO_INCREMENT PRIMARY KEY, name VARCHAR(50), email VARCHAR(100) UNIQUE, salary INT DEFAULT 30000);
```

CHECK : CHECK is used to restrict values in a column based on a condition.

Top 5 Most Asked Interview Questions

Question 1 : Create an employees table where salary must be greater than 0.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), salary INT CHECK(salary > 0));
```

Question 2 : Create an employees table where age must be at least 18.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), age INT CHECK(age >= 18));
```

Question 3 : Create an employees table where salary must be between 30000 and 100000.

```
CREATE TABLE employees (empid INT PRIMARY KEY, salary INT CHECK(salary BETWEEN 30000 AND 100000));
```

Question 4 : Create an employees table where rating must be between 1 and 5.

```
CREATE TABLE employees (empid INT PRIMARY KEY, rating INT CHECK(rating BETWEEN 1 AND 5));
```

FOREIGN KEY : A FOREIGN KEY is a column that creates a relationship between two tables and ensures that values exist in the referenced table.

Top 5 Most Asked Interview Questions

Question 1 : Create a departments table with dept_id as the Primary Key.

```
CREATE TABLE departments (dept_id INT PRIMARY KEY, department_name  
VARCHAR(50));
```

Question 2 : Create an employees table with dept_id as a Foreign Key.

```
CREATE TABLE employees (empid INT PRIMARY KEY, name VARCHAR(50), dept_id  
INT, FOREIGN KEY(dept_id) REFERENCES departments(dept_id));
```

Question 3 : Create a students table linked to a courses table using a Foreign Key.

```
CREATE TABLE students (student_id INT PRIMARY KEY, name VARCHAR(50),  
course_id INT, FOREIGN KEY(course_id) REFERENCES courses(course_id));
```

28. INSERT : INSERT is used to add new records into a table.

Top 5 Most Asked Interview Questions

Question 1 : Insert a single employee record.

```
INSERT INTO employees VALUES (1,'Keerti',50000);
```

Question 2 : Insert a record using specific columns.

```
INSERT INTO employees(empid,name,salary) VALUES (2,'Rahul',60000);
```

Question 3 : Insert a record into an AUTO_INCREMENT table.

```
INSERT INTO employees(name,email) VALUES ('Keerti','keerti@gmail.com');
```

29. UPDATE : UPDATE is used to modify existing records in a table.

Top 5 Most Asked Interview Questions

Question 1 : Update the salary of an employee.

```
UPDATE employees SET salary = 70000 WHERE empid = 2;
```

Question 2 : Update multiple columns in a single query.

```
UPDATE employees SET name = 'Rahul Kumar', salary = 70000 WHERE empid = 2;
```

Question 3 : Update all rows in a table.

```
UPDATE employees SET salary = 50000;
```

30. DELETE : DELETE is used to remove records from a table.

Top 5 Most Asked Interview Questions

Question 1 : Delete an employee using empid.

DELETE FROM employees WHERE empid = 1;

Question 2 : Delete an employee using name.

DELETE FROM employees WHERE name = 'Keerti';

Question 3 : Delete all employees from the HR department.

DELETE FROM employees WHERE department = 'HR';

Question 4 : Delete all records from a table.

DELETE FROM employees;

31. ALTER TABLE : ALTER TABLE is used to modify the structure of an existing table.

Top 5 Most Asked Interview Questions

Question 1 : Add a new column age to the employees table.

ALTER TABLE employees ADD age INT;

Question 2 : Change the datatype of salary from INT to BIGINT.

ALTER TABLE employees MODIFY COLUMN salary BIGINT;

Question 3 : Remove the salary column from the employees table.

ALTER TABLE employees DROP COLUMN salary;

Question 4 : Rename the age column to employee_age.

ALTER TABLE employees RENAME COLUMN age TO employee_age;

Question 5 : Rename the employees table to employees_details.

ALTER TABLE employees RENAME TO employees_details;

31. TRUNCATE TABLE : TRUNCATE TABLE removes all rows from a table but keeps the table structure.

Top 5 Most Asked Interview Questions

Question 1 : Remove all records from the employees table.

TRUNCATE TABLE employees;

32 . DROP TABLE : DROP TABLE permanently removes a table along with its structure and data.

Top 5 Most Asked Interview Questions

Question 1 : Delete the employees table completely.

DROP TABLE employees;

Question 2 : Delete the students table completely.

DROP TABLE students;

Question 3 : Delete the products table completely.

DROP TABLE products;

SPOTIFY MUSIC ANALYSIS PROJECT INTERVIEW QUESTIONS & ANSWERS

1.Explain your Spotify Music Analysis project.

I developed a Spotify Music Analysis project using MySQL. I analyzed song data to identify trends in popularity, danceability, energy, and duration. I used SQL concepts such as SELECT, GROUP BY, aggregate functions, subqueries, and CASE statements to generate insights from the dataset. The project helped me improve my SQL and data analysis skills.

2.Which dataset did you use?

I used the Spotify Dataset from Kaggle.

3.Why did you choose Spotify data?

I chose Spotify data because it contains real-world music information and several attributes such as popularity, energy, danceability, and duration. It was a good dataset for practicing SQL and data analysis.

4.Which database did you use?

I used MySQL and MySQL Workbench.

5.How many records were analyzed?

I analyzed 253 records.

6.What SQL concepts did you use?

-SELECT -WHERE

-ORDER BY -LIMIT

- COUNT() -AVG()
- MIN() -MAX()
- GROUP BY
- Subqueries -CASE Statements
- CASE -Statements

7.What was the most popular song?

The most popular song in the dataset was Weather Bird.

8.What was the highest popularity score?

The highest popularity score was 42.

9.What was the average song duration?

The average song duration was 3.81 minutes.

10.Which year had the highest average popularity?

The year 1922 had the highest average popularity.

11.What is GROUP BY?

GROUP BY is used to group rows with the same values and perform calculations on each group.

Example:Average popularity for each year.

12.Why did you use GROUP BY in your project?

I used GROUP BY to analyze data year-wise. For example, I calculated average popularity and average energy for each year.

13.What is a subquery?

A subquery is a query written inside another query. It uses the result of one query inside another query.

14.Which query used a subquery?

```
SELECT name, artists, popularity
FROM spotify_data
WHERE popularity >
(
SELECT AVG(popularity)
FROM spotify_data
);
```

This query finds songs with popularity greater than the average popularity.

15.What is a CASE statement?

CASE is used to apply conditions and return different results based on those conditions. It works like IF-ELSE in programming.

16.Why did you use CASE?

I used CASE to classify songs into Popular, Moderate, and Low categories based on popularity scores.

17.Difference between WHERE and HAVING?

WHERE filters rows before grouping.

HAVING filters groups after GROUP BY.

18.Difference between COUNT(*) and COUNT(column)?

COUNT(*) counts all rows.

COUNT(column) counts only non-NULL values in a column.

19.Difference between MAX() and ORDER BY DESC LIMIT 1?

MAX() returns only the highest value.

ORDER BY DESC LIMIT 1 returns the complete row having the highest value.

20.What insights did you generate?

- Total songs analyzed: 253
- Most popular song: Weather Bird
- Highest popularity score: 42
- Average song duration: 3.81 minutes
- Most danceable song: I Must Have That Man
- Highest energy song: Korkma Sönmez
- Year with highest average popularity: 1922
- All songs in the dataset were non-explicit.

21.What challenges did you face during the project?

Initially, I faced difficulties while importing the Spotify dataset into MySQL. The dataset was not importing correctly, and I encountered issues related to file paths, CSV formatting, and incomplete record imports. I checked the file format, corrected the CSV file, and re-imported the dataset successfully. This helped me understand data import processes and troubleshoot database-related issues.